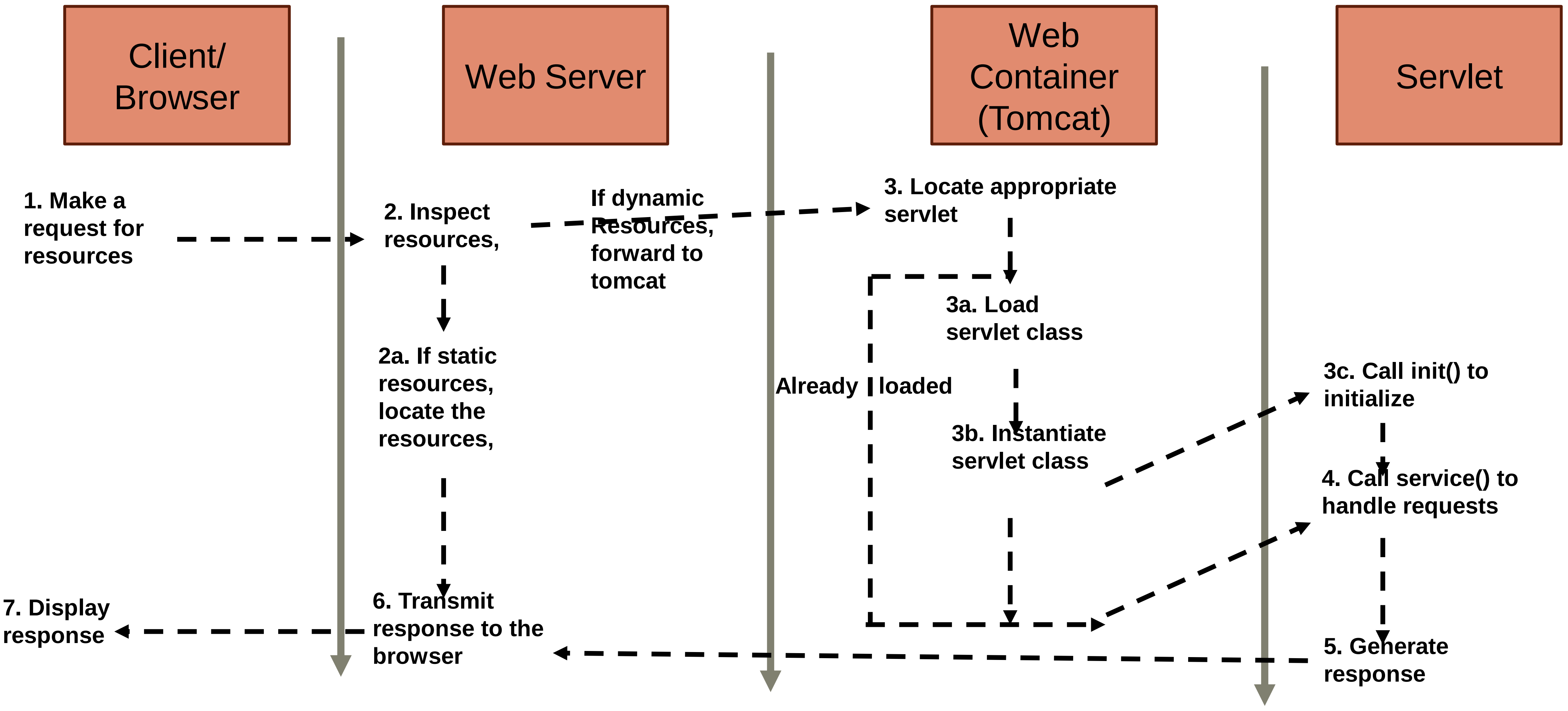
SIT218/SIT738
Secure coding unit,
T2 2023

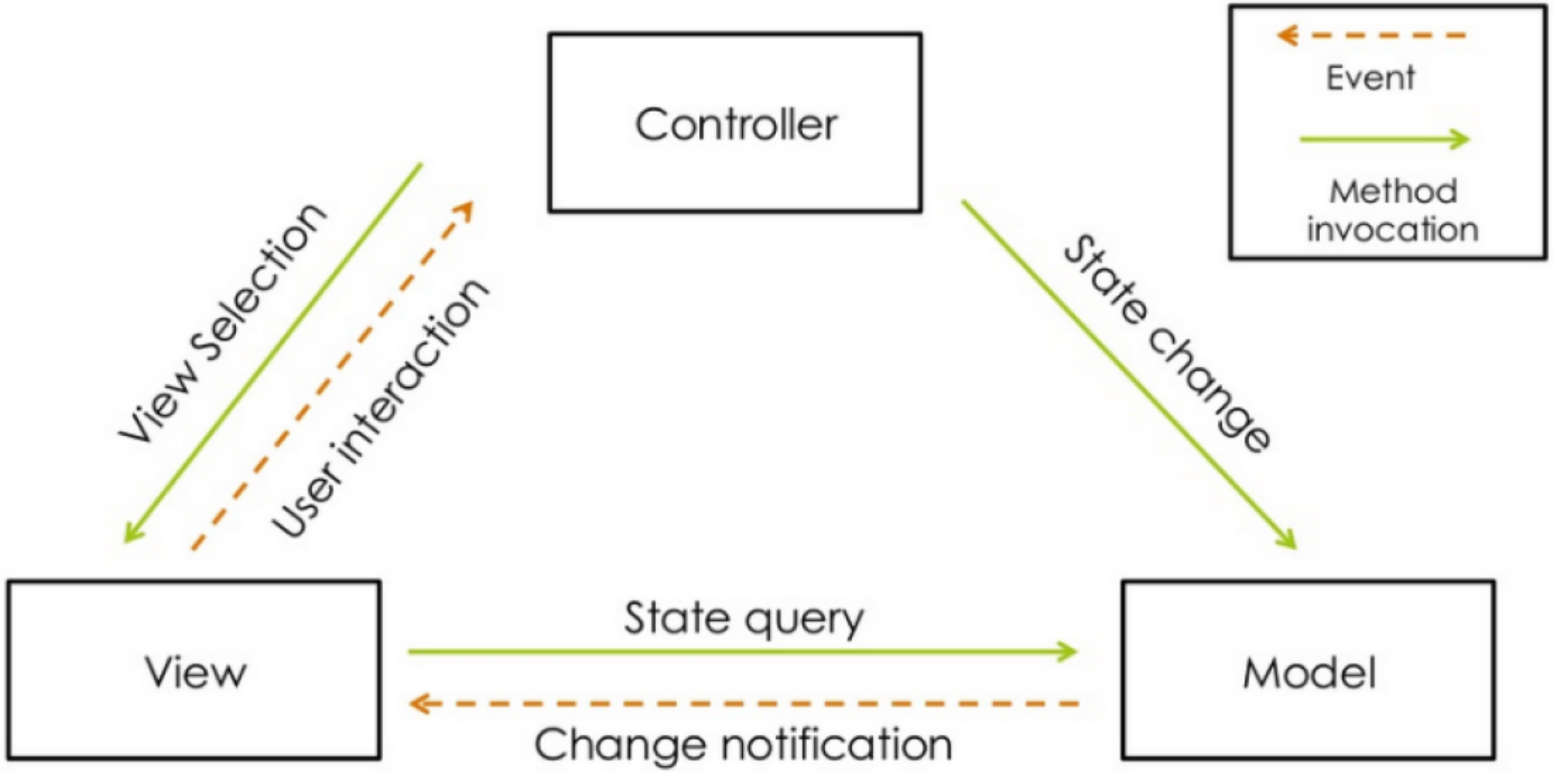
Dr Shamsul Huda

Model view controller-Spring Framework

SIT218/SIT738 Requests from browser to webserver and response to the client



SIT218/SIT738 Model view controller-Spring Framework

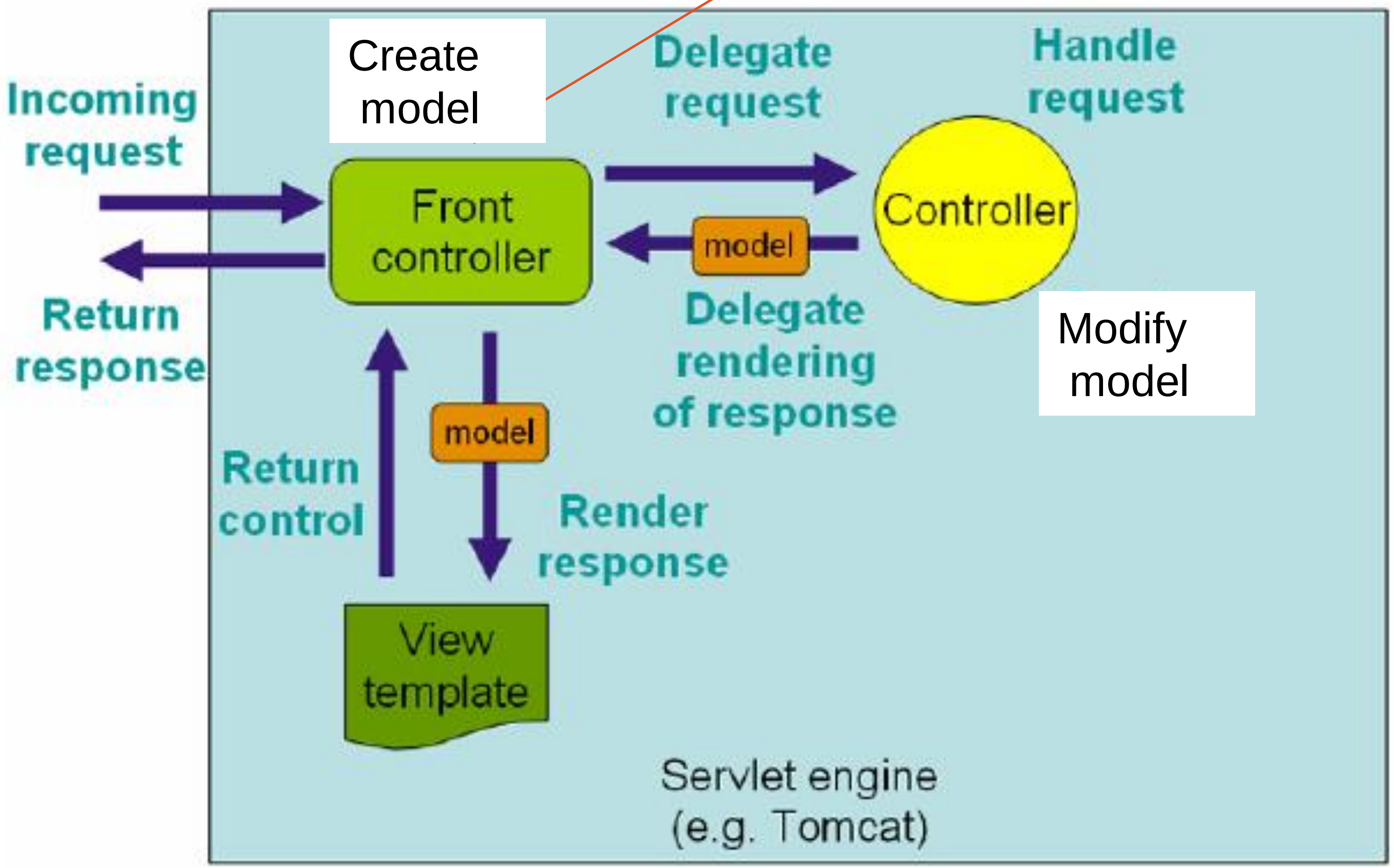


A typical MVC is presented using three main components

- **The models - A model contains the data of the application, such as a database, web service, etc.**
- **The views - A view represents the provided information in a particular format, such as JSP, Thymelead, Groovy, Velocity, etc.**
- **The controllers - A controller contains the business logic of an application. This business logic is usually implemented on the server-side.**

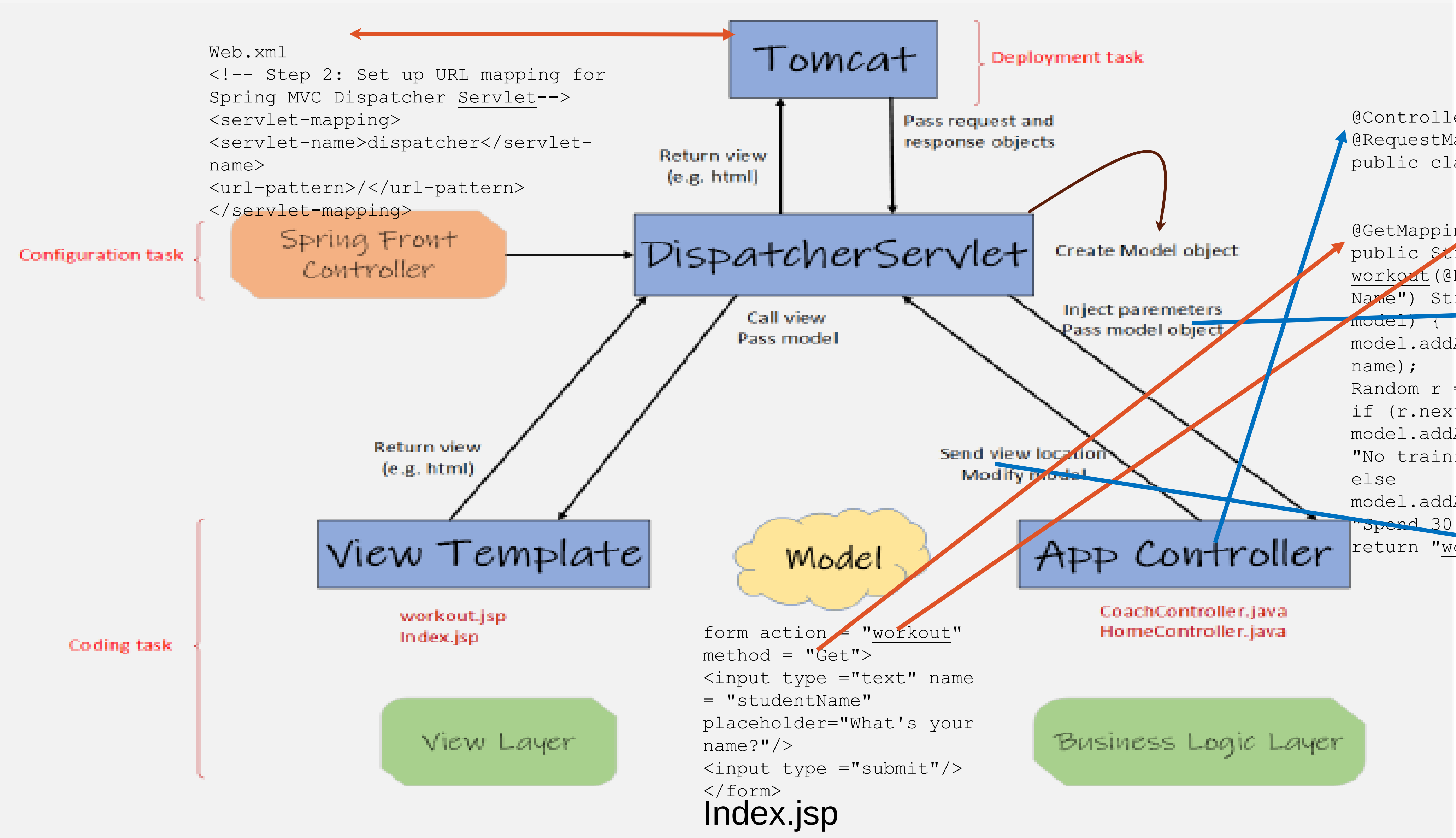
SIT218/SIT738 Model view controller

Dispatcher servlet,
create model object



Request-response flow in MDC

SIT218/SIT738 Model view controller-Spring Framework




```
Web.xml
<!-- Step 2: Set up URL mapping for
Spring MVC Dispatcher Servlet-->
<servlet-mapping>
<servlet-name>dispatcher</servlet-
name>
<url-pattern>/</url-pattern>
</servlet-mapping>
```

```
@Controller
@RequestMapping("/workout")
public class CoachController {

@GetMapping
public String
workout(@RequestParam("student
Name") String name, Model
model) {
model.addAttribute("name",
name);
Random r = new Random();
if (r.nextBoolean())
model.addAttribute("message",
"No training today");
else
model.addAttribute("message",
"Spend 30 min running");
return "workout";
```

```
form action = "workout"
method = "Get">
<input type ="text" name
= "studentName"
placeholder="What's your
name?"/>
<input type ="submit"/>
</form>
```

Index.jsp

```
</body>
</html>
```

SIT218/SIT738 Model view controller-Spring Framework

1. Okay, let's review the Spring MVC request flow in short:
2. When we enter a URL in the browser, the request comes to the tomcat server. It understands it’s JSP pages, then compiles JSP pages to servlet classes. Then passes the requests to the Dispatcher servlet. The Dispatcher servlet acts as a centralized entry point to the web application.
3. The Dispatcher servlet creates a model determines a suitable controller based on the request mapping. It passes the model object to the controller also.
4. The controller method updates the objects in the model and returns the logical view name and updated model to the Dispatcher servlet.
5. The Dispatcher servlet call the view based on the view location in configuration file and passes the model's data to that view.
6. View furnishes the dynamic values in the web page using the model's data and renders the final web page and returns that web page to the Dispatcher servlet.
7. At the end, the Dispatcher servlet returns the final rendered page as a response to the browser.

SIT218/SIT738 Model view controller-Spring Framework

```

<web-app>

    <servlet>
        <servlet-name>example</servlet-name>
        <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
        <load-on-startup>1</load-on-startup>
    </servlet>

    <servlet-mapping>
        <servlet-name>example</servlet-name>
        <url-pattern>/example/*</url-pattern>
    </servlet-mapping>

</web-app>
```

All requests starting with /example will be handled by dispatcher servlet

SIT218/SIT738 Model view controller-Spring Framework

```xml
<!-- Step 1: Configure Spring MVC Dispatcher Servlet -->
<absolute-ordering />

<servlet>
 <servlet-name>dispatcher</servlet-name>
 <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
 <init-param>
  <param-name>contextConfigLocation</param-name>
  <param-value>/WEB-INF/spring-configuration.xml</param-value>
 </init-param>
 <load-on-startup>1</load-on-startup>
</servlet>

<!-- Step 2: Set up URL mapping for Spring MVC Dispatcher Servlet -->
<servlet-mapping>
 <servlet-name>dispatcher</servlet-name>
 <url-pattern>/</url-pattern>
</servlet-mapping>
```

/WEB-INF/spring-configuration.xml.

All requests starting with / pattern will be handled by dispatcher servlet